

Dynamic sparse PCA: a dimensional reduction method for sensor data in virtual metrology

Tianhui Wang^a, Yifan Xie^a, Young-Seon Jeong^{b,*}, Myong K. Jeong^{a,*}

^a Department of Industrial and Systems Engineering, Rutgers University, Piscataway, 08854, NJ, USA

^b Department of Industrial Engineering and Interdisciplinary Program of Arts & Design Technology, Chonnam National University, Gwangju, 61186, South Korea

ARTICLE INFO

Keywords:

Dimensional reduction
Semiconductor manufacturing
Sparse PCA
Virtual metrology

ABSTRACT

Virtual metrology (VM) is the ideal technique for predicting the quality of every wafer and reducing overall inspection costs in semiconductor manufacturing. The data from numerous monitoring sensors in semiconductor production poses challenges in terms of data dimensionality. Thus, dimensionality reduction techniques are required before building the prediction model in VM. Additionally, to reduce the sensor cost, it is crucial to determine the sensors that contribute significantly to the output of the prediction model. Therefore, in this paper, we propose a dimensionality reduction technique called dynamic sparse principal component analysis to establish global sparsity in the first few principal components. The proposed method seeks extracted features that involve a limited number of sensors in the VM to reduce sensor cost and increase the interpretability of the features while reasonably capturing data variability. The experimental results demonstrate that the proposed method is not sensitive to the scale of the problem and can solve the large-scale high-dimensional problems that are commonly encountered in semiconductor manufacturing.

1. Introduction

1.1. Background

Semiconductor manufacturing involves hundreds of complex processes. The average cycle time of a batch of wafers is at least 12 weeks. To check the quality of the finished wafers, the wafer acceptance test (WAT) is widely used (Pan et al., 2011). In this test, a large amount of data related to wafer performance is collected. However, these WAT data are end-of-line metrology data, that can be obtained only after production. Defective wafers end up being discarded, resulting in wastage of material and production capacity (Adly et al., 2015). To monitor wafer quality throughout the process, in-line metrology (ILM) is adopted to acquire critical data after vital steps, such as thermal process, photolithography, etching, implant, deposition, and polishing. Nonetheless, ILM is physical metrology, and it is expensive and time-consuming. Additionally, it is available only for a limited number of sampled wafers.

Virtual metrology (VM) represents the ideal approach to reduce inspection costs and ensure the product quality of an entire lot of wafers. VM is widely used in many manufacturing domains to evaluate the performance of final products (Nguyen et al., 2019). In semiconductor manufacturing, VM is used to predict the quality of every wafer by

training a machine learning model that uses the sensor data collected during production and the output data measured after each process.

1.2. Literature survey

Over the past several decades, many VM schemes have been proposed by researchers. Principal component analysis (PCA) and partial least squares (PLS) are conventionally used dimensionality reduction techniques in VM systems (Kang et al., 2009). Zeng and Spanos (2009) proposed VM models in which they used PCA and kernel PCA to reduce input dimensionality and the principal component regression model to make predictions over the etching process. Arima et al. (2019) developed a three-phase VM model in which they used PCA and the least absolute shrinkage and selection operator (LASSO) for automatic variable extraction to determine multi-dimensionality, which expresses the connections among the input variables and the condition of the production machine and predicts wafer quality after the processes. Ren and Ni (2021) established a local monitoring model by using the multiway principal component analysis method to project the original variables from a high-dimensional to a low-dimensional feature space. The feature space contained only a few features as the input variables for the statistical model of the batch process data.

* Corresponding authors.

E-mail addresses: tianhui.wang@rutgers.edu (T. Wang), yifan.xie@rutgers.edu (Y. Xie), young.jeong@jnu.ac.kr (Y.-S. Jeong), mjeong@soe.rutgers.edu (M.K. Jeong).

<https://doi.org/10.1016/j.eswa.2024.123995>

Received 17 August 2023; Received in revised form 6 April 2024; Accepted 14 April 2024

Available online 17 April 2024

0957-4174/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Building upon the cornerstone dimensionality reduction techniques such as PCA and PLS, the exploration of alternative methods has brought significant advancements to the field. Notably, singular value decomposition (SVD) has emerged as a pivotal enhancement, offering a refined methodology for data decomposition. This technique, eloquently proposed by [Abdi \(2007\)](#), streamlines the transformation of datasets into singular vectors and values, thereby enabling a more coherent representation of data in reduced dimensional spaces. The domain of dimensionality reduction has witnessed further expansion with the advent of innovative approaches. Among these, the discriminative and geometry-preserving adaptive graph embedding (DG-PAGE) proposed by [Gou et al. \(2023\)](#) represents a significant leap forward. By amalgamating manual and adaptive graph constructions, DGPAGE adeptly captures and conserves the essential properties of high-dimensional data within a compact subspace. Similarly, the constrained singular value decomposition (CSVD) introduced by [Guillemot et al. \(2019\)](#) presents a novel perspective by weaving constraints such as sparsity and orthogonality into the fabric of the decomposition process. This innovation paves the way for its application across a spectrum of domains. These developments underscore a vibrant evolution in the realm of dimensionality reduction. They mark a departure from conventional methodologies towards more flexible and sophisticated techniques, thereby expanding the scope for research and practical applications in VM systems and beyond.

To build an efficient VM system, the following capabilities are preferred ([Susto & Beghi, 2013](#)). First, since the number of original variables obtained from the sensor data can be in the thousands, to build a VM model, dimensionality reduction techniques are used to decrease the number of input variables. Feature extraction is a common data-reduction technique; it transforms raw data into a set of features with information that serves as building blocks for information processing, learning, and applications ([Xia et al., 2021](#)). Additionally, the interpretability of the input variables is emphasized by engineers. Therefore, it is desirable to extract features by using the data obtained from a limited number of sensors. Furthermore, the semiconductor domain is rapidly updated, new products and procedures are introduced to the manufacturing system on a monthly basis, and a VM model with reasonable computation times is required. Meanwhile, among the sensors that collect thousands of variables, not all sensors are essential for obtaining the desired prediction result. The selection of informative features can significantly improve learning accuracy and the comprehension of results ([Li et al., 2022](#)). Therefore, the selection of a limited number of essential sensors is critical for increasing the prediction accuracy and reducing the cost of sensors. As mentioned above, the data obtained from hundreds of monitoring sensors in semiconductor manufacturing are typically high-dimensional. This data remains underutilized directly due to its high-frequency and high-dimensional nature, making analysis a challenge ([Corallo et al., 2021](#)). Additionally, a minority of these variables significantly influence the output of the prediction model. Both variable selection and feature extraction techniques are needed to process the raw data to obtain accurate predictions and interpretable inputs in the VM model. However, in the VM models reported in the literature, PCA is only used as a feature extraction method to decrease the number of input variables in the VM model.

PCA was developed by [Pearson \(1901\)](#), and it is a widely used dimensionality reduction and feature extraction technique, with numerous applications in the science and engineering domain [Jolliffe \(2002\)](#). PCA extracts the principal components (PCs), which are linear combinations of the original variables that account for the maximal variance in the transformed variables. Given a data matrix or covariance matrix, PCA can be implemented using singular value decomposition (SVD) or eigenvalue decomposition, respectively ([Berk & Bertsimas, 2019](#); [Qiu et al., 2023](#)). Owing to PCA implementation, the original data can be compressed with minimum information loss by reserving the first few PCs that capture the most amount of information. Moreover, the obtained PCs are uncorrelated, which is a valuable property

from the perspectives of interpretation and subsequent statistical analysis ([d'Aspremont et al., 2004](#)). One of the objectives of PCA is to increase the variance of the projected data by analyzing a series of projections. These projections form a low-dimensional linear subspace, allowing us to successfully capture the data structure in the initial input space ([Kwak, 2014](#)).

Nonetheless, PCA has several drawbacks. For instance, the loading vectors lack sparseness. That is, each PC consists of all the original variables. In semiconductor manufacturing, the original variables obtained from different sensors have various meanings. Consequently, if all the PCs include all the original variables, the meanings of new features would be ambiguous and lack interpretability.

To overcome this limitation of PCA, many methods have been proposed. [Zou and Xue \(2018\)](#) provided a selective overview of several such methods. Rotation techniques have been developed to interpret the PCs ([Jolliffe, 1995](#)). The simple principal components method restricts the choice of loading values to a narrow set of selected numbers such as 0, 1 and -1 ([Vines, 2000](#)). This simple method is suitable only in specific cases. The simple thresholding method adjusts the loading vectors if the absolute value of an element is lower than a threshold to achieve sparse loadings ([Cadima & Jolliffe, 1995](#)). This ad hoc approach is commonly used in practice, but its results are not always accurate.

Based on the formulation of PCA, [Jolliffe et al. \(2003\)](#) proposed a new formulation called SCoTLASS by adding an l_1 constraint to obtain sparse loading vectors. The sparseness of the loading vectors is controlled by a tuning parameter t , which does not have a direct explicit functional relationship with sparseness. In addition, the examples presented in their paper indicate that the loading vectors acquired using the SCoTLASS method are not sufficiently sparse when a high percentage of explained variance (EV) is required ([Jolliffe et al., 2003](#)). Moreover, the computational cost of the SCoTLASS is high, which makes it impractical to apply this method to high-dimensional data.

[Zou et al. \(2006\)](#) proposed a sparse principal component analysis (SPCA) method by recasting PCA as an elastic-net regression problem with the LASSO penalty to reduce computational time for high-dimensional data. [d'Aspremont et al. \(2004\)](#) relaxed cardinality constraint to maximize the variance of the transformed variables formulation and derived a semi-definite programming relaxation scheme as a direct sparse PCA formulation. [Moghaddam et al. \(2005\)](#) considered a discrete formulation that maximized the variance of the transformed variables and solved the resulting combinatorial optimization problem by using an exact algorithm in conjunction with a branch-and-bound method. Additionally, they used a bi-directional greedy search method to initialize the solution of the exact algorithm. [Liu et al. \(2018\)](#) imposed a joint $\ell_{2,1}$ norm penalty to achieve row-wise sparsity to reduce the number of input variables used in extracted PCs. The formulation was based on SPCA; consequently, its drawbacks were identical to those of SPCA. The method required a regularization parameter that needed to be tuned to achieve the desired sparsity. [Seghouane et al. \(2019\)](#) proposed an adaptive block sparse algorithm ensuring consistent sparsity patterns across all principal components, which enhances feature selection performance in image processing tasks. [Berk and Bertsimas \(2019\)](#) introduced a branch-and-bound algorithm designed to address the sparse PCA formulation, specifically incorporating the ℓ_0 norm constraint. They adeptly employed first-order techniques to ascertain feasible solutions and strategically obtained upper and lower bounds. [Richtárik et al. \(2021\)](#) explored eight distinct optimization formulations with the primary objective of deriving a solitary sparse loading vector. These formulations emerge from the interplay of three essential modeling elements: the utilization of either the ℓ_2 or ℓ_1 norms for variance measurement, in conjunction with the application of ℓ_0 or ℓ_1 norms as sparsity-inducing factors. These norms are implemented in two distinct modes, either as constraints or penalties. Subsequently, the authors adopt the alternating maximization method to address the reformulations effectively. [Qiu et al. \(2023\)](#) created gradient-based and projection-free algorithms to address an unconstrained convex

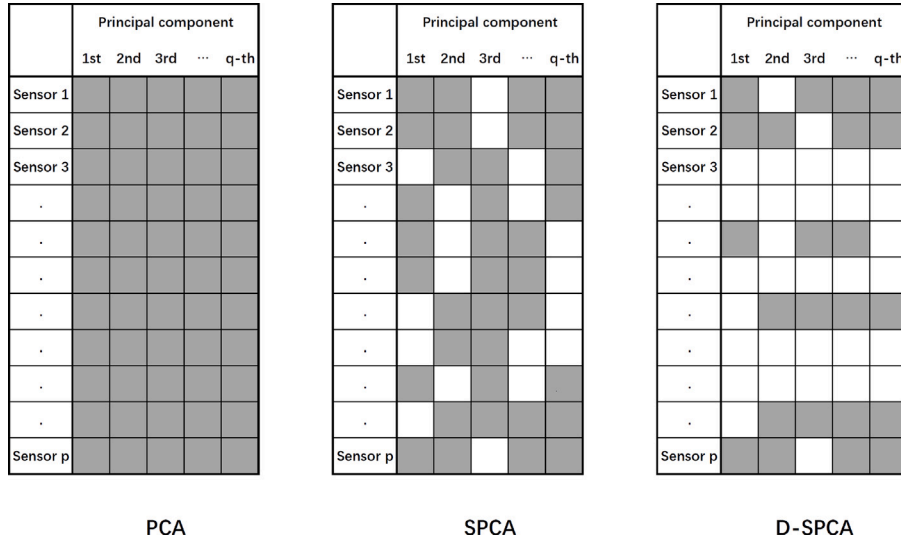


Fig. 1. The difference in loading vector between PCA, sparse PCA, and D-SPCA. The gray cells represent the nonzero loading vectors in each principal component, and the white cells are zero.

optimization problem, aiming to derive initial values for non-convex methodologies. Subsequently, they extended the unconstrained convex framework to an online context, accommodating substantial-scale or high-dimensional datasets.

1.3. Motivation

The aforementioned sparse PCA methods use all the original input variables when more PCs are considered. As shown in Fig. 1, sparse PCA, in contrast to traditional PCA, produces loading vectors with sparser components. However, it merely considers sparsity within each individual PC, necessitating the inclusion of data from all sensors in the final constructed VM model. On the other hand, the loading vectors obtained through the proposed sparse PCA approach on the right-hand side exhibit sparsity across all principal components, which aligns with the preferred outcomes within the VM model. In real-world applications, the cost of data acquisition is often non-negligible, and especially in the engineering domain, each input variable corresponds to a sensor.

In this paper, we propose a novel sparse PCA approach named the dynamic sparse PCA (D-SPCA) method. The proposed method includes a dynamic sparse algorithm that aims to achieve global sparsity across the first few PCs, thereby ensuring the extracted features utilized as input variables in the VM model encompass a restricted set of sensors. Compared with existing sparse PCA methods, the reduction in globally used variables achieved by D-SPCA leads to cost savings in sensors and enhances interpretability for the VM model, making it more practical for real-world applications. According to numerical results obtained from synthetic and real-world examples, the contribution and benefits of the proposed method are as follows:

1. The D-SPCA method seeks the global sparsity in the first few PCs, we use fewer variables in total while capturing similar data variability as the other sparse PCA methods.
2. D-SPCA method can provide near-optimal solutions within a reasonable timeframe for large-scale problems with thousands of variables.
3. Owing to the integrated variable selection technique used in the dynamic sparse algorithm, fewer variables are involved in the VM model, which decreases sensor cost, and the prediction performance of the proposed VM model is similar to that of the benchmark methods.
4. The sparse loading vector ensures the interpretability of each extracted feature. The backward elimination algorithm weakens the nesting effect of the greedy forward selection method.

Table 1

Table of notation.

$\mathbf{a}_i$	The loading vector of the i th PC
$\mathcal{L}_j$	A set of used variables in the first j PCs
$\mathcal{P}$	A set of input variables
$\mathcal{U}_j$	A set of used variables in the j th PC
$\mathcal{V}_j$	A set of variance values in the j th PC
Σ	The original covariance matrix of all input variables
Σ_j	The deflated covariance matrix at the j th PC
$\mathbf{x}$	A random vector with p variables $\mathbf{x} = [x_1, x_2, \dots, x_p]^T$
x_i	The i th input variable
$\mathbf{X}$	The input data matrix, $\mathbf{X} \in \mathbb{R}^{n \times p}$
$\mathbf{z}$	A transformed variable (extracted feature)
z_i	The i th principal component
$\mathbf{Z}$	The transformed input data matrix, $\mathbf{Z} \in \mathbb{R}^{n \times q}$
$\ \cdot\ _0$	ℓ_0 norm (the number of non-zero elements in a vector)
$O(\cdot)$	Computational complexity
$Var(\cdot)$	Variance of the variable
$Cov(\cdot)$	Covariance of the variable
K_j	Number of nonzero elements required in the j th PC
k_j	Number of variables in the j th PC
n	Number of observations in the input data
M	Number of sensors
p	Number of variables in the input data
q	Number of PCs (features) to be extracted

1.4. Paper organization

The rest of this paper is organized as follows: Section 2 briefly reviews the PCA and sparse PCA formulations. In Section 3, we present a variant sparse PCA formulation and the proposed D-SPCA algorithm. The numerical results obtained by applying the proposed algorithm to synthetic and real-world data are presented in Section 4. Finally, Section 5 provides our conclusions and an outline for future work.

2. Related works

As mentioned in Section 1.2, it is important to consider dimensionality reduction, variable selection, interpretability of the input variables, and computation time as essential aspects when building the VM model. PCA and its variants are popular techniques for achieving these goals. In this section, we briefly introduce the general definitions of PCA and sparse PCA that are used in the VM model. Table 1 provides an explanation for the notations used in the paper.

2.1. PCA

Let $\mathbf{x} = [x_1, x_2, \dots, x_p]^\top$ denote a random vector with the covariance matrix $\Sigma_{p \times p}$, where x_i represents the i th input variable and p represents the number of variables in $\mathbf{x}$. Given a vector $\mathbf{a} = [a_1, a_2, \dots, a_p]$, a transformed variable z can be derived by the linear combinations of original input variables as follows:

$$z = \mathbf{a}^\top \mathbf{x} = a_1 x_1 + a_2 x_2 + \dots + a_p x_p.$$

PCA seeks the loading vector $\mathbf{a}_i$ that maximizes the variance of the transformed variables, also referred to as principal components. The variance and covariance of the PCs can be calculated as follows:

$$Var(z_i) = \mathbf{a}_i^\top \Sigma \mathbf{a}_i \quad \text{for } i = 1, 2, \dots, p,$$

$$Cov(z_i, z_j) = \mathbf{a}_i^\top \Sigma \mathbf{a}_j \quad \text{for } i, j = 1, 2, \dots, p, i \neq j.$$

The first principal component z_1 can be calculated as $z_1 = \mathbf{a}_1^\top \mathbf{x}$, where $\mathbf{a}_1$ is the loading vector that provides the maximum variance for z_1 . The formulation for solving the loading vector of the first PC is presented as the following optimization problem.

$$\begin{aligned} \max \quad & \mathbf{a}_1^\top \Sigma \mathbf{a}_1 \\ \text{s.t.} \quad & \mathbf{a}_1^\top \mathbf{a}_1 = 1 \end{aligned} \quad (1)$$

The objective function, namely the variance of z_1 , is constrained by $\mathbf{a}_1^\top \mathbf{a}_1 = 1$, which limits the length of vector $\mathbf{a}_1$ and thereby ensures that the variance of z_i does not grow indefinitely, eliminating the indeterminacy.

The formulations of the second and the subsequent PCs can be expressed as problem (2), and they are similar to the one for the first PC, except for an additional constraint that guarantees the loading vector of the following PC is orthogonal to all previous loading vectors.

$$\begin{aligned} \max \quad & \mathbf{a}_i^\top \Sigma \mathbf{a}_i \\ \text{s.t.} \quad & \mathbf{a}_i^\top \mathbf{a}_j = 0 \\ & \mathbf{a}_i^\top \mathbf{a}_j = 0 \end{aligned} \quad (2)$$

where $i, j = 1, 2, \dots, p$ and $i \neq j$. The i th PC z_i is determined using the loading vector $\mathbf{a}_i$.

Given a total of p variables, PCA extracts q features or PCs that explain most of the variance in the original data. However, each PC is a linear combination of all variables, which is not conducive for interpreting the extracted features. Thus, sparse PCA is developed to generate sparse loading vectors. The advantage of using the sparse loading vectors is that each feature consists of only a small number of variables.

2.2. Sparse PCA

Through the addition of a constraint that restricts the number of nonzero elements in the loading vector of the original PCA formulation, the first sparse PC can be determined by solving the following optimization problem (3):

$$\begin{aligned} \max \quad & \mathbf{a}_1^\top \Sigma \mathbf{a}_1 \\ \text{s.t.} \quad & \mathbf{a}_1^\top \mathbf{a}_1 = 1 \\ & \|\mathbf{a}_1\|_0 \leq K_1 \end{aligned} \quad (3)$$

where $\|\mathbf{a}_1\|_0$ in the constraint denotes the ℓ_0 norm, meaning that $\|\mathbf{a}_1\|_0 = |a_{11}|^0 + |a_{12}|^0 + \dots + |a_{1p}|^0$. K_1 is the number of nonzero elements in the first sparse PC, it guarantees that the number of nonzero elements in the first PC is less than the preset value K_1 .

For the subsequent sparse PCs, after the addition of a similar ℓ_0 norm constraint, the formulation can be expressed as the problem (4).

$$\begin{aligned} \max \quad & \mathbf{a}_i^\top \Sigma \mathbf{a}_i \\ \text{s.t.} \quad & \mathbf{a}_i^\top \mathbf{a}_j = 0 \\ & \mathbf{a}_i^\top \mathbf{a}_j = 0 \\ & \|\mathbf{a}_i\|_0 \leq K_i \end{aligned} \quad (4)$$

where $i, j = 1, 2, \dots, p$ and $i \neq j$. K_i is the number of nonzero elements in the i th sparse PC.

Although the above-expressed optimization problems of sparse PCA are non-convex and NP-hard, compared to the other sparse PCA methods in the literature, this formulation has a significant advantage. The parameter K in this formulation is directly related to sparseness. Moghaddam et al. (2005) provided a sound theoretical basis for selecting K .

Both the sparse PCA with the ℓ_0 norm constraint and the other methods mentioned in the literature tend to use all the original input variables when many PCs are considered. Although a few existing sparse PCA formulations consider global sparsity, which reduces the number of original variables used in PCs, they fail to remain sparse for individual PCs. The greedy method reported in the literature only adds variables to the nonzero list, resulting in a nesting effect, and it ignores several potential reasonable solutions with strong correlations between variables (d'Aspremont et al., 2008). The dynamic concept proposed herein guarantees a wider search space that can yield potentially better solutions ignored by the greedy method.

Fig. 2 illustrates all possible combinations for any data set with $p = 5$ variables when the number of selected nonzero variables is K , where $K = 1, 2, \dots, p$. The enumeration method searches all the combinations to identify the optimal one. The size of the search space is $\binom{p}{K}$. The combinations circled in green boxes represent the search space of the greedy method, which adds the variable that maximizes the explained variance to the variable list one by one until the number of added variables reaches a predetermined value K . The solutions circled in red are the best solutions for each K . However, once the variable providing the largest variance is selected, the subsequent solutions must rely on the previous solutions. The size of the search space is $\sum_{k=K}^p \binom{p}{k}$. The solutions circled in the yellow boxes are the additional possible solution other than those identified using the greedy method, and they are discovered by means of backward search, which is introduced in the following section.

3. Dynamic sparse PCA method

In contrast to the enumeration method that searches among $\binom{p}{K}$ possible combinations, the D-SPCA method outlined in Algorithm 1 seeks a set of variables and the corresponding loading vectors that maximize the EV of each PC. The set of selected variables is determined using the forward variable selection (FVS) method presented in Algorithm 2 and the backward variable elimination (BVE) method presented in Algorithm 3.

3.1. Dynamic sparse PCA formulation

Given a covariance matrix $\Sigma_{p \times p}$ in descending eigenvalue order formed by a set of input variables $\mathcal{P} = \{x_i | i = 1, 2, \dots, p\}$, let K_j be the desired number of nonzero variables in j th sparse PC and M be the number of sensors available. The D-SPCA method returns the sparse loading vectors in a matrix format $\mathbf{a}_{p \times q}$ that represents a near-optimal solution to the following optimization problem (5):

$$\begin{aligned} \max \quad & \mathbf{a}_j^\top \Sigma_j \mathbf{a}_j \\ \text{s.t.} \quad & \mathbf{a}_j^\top \mathbf{a}_j = 1 \\ & \|\mathbf{a}_j\|_0 \leq K_j \\ & \|\mathcal{L}_j\|_0 \leq M \end{aligned} \quad (5)$$

where $\mathbf{a}_j$ is the j th column of $\mathbf{a}_{p \times q}$ and $j = 1, 2, \dots, q$, Σ_j is the deflated covariance matrix after extracted $(j-1)$ th principal component, and it can be calculated using Eq. (6) (Mackey, 2008), where $\mathbf{a}_{j-1}^\top \Sigma_{j-1} \mathbf{a}_{j-1}$ is a scalar, and $\mathbf{a}_{j-1} \mathbf{a}_{j-1}^\top$ is $p \times p$ matrix.

$$\Sigma_j = \Sigma_{j-1} - \mathbf{a}_{j-1}^\top \Sigma_{j-1} \mathbf{a}_{j-1} (\mathbf{a}_{j-1} \mathbf{a}_{j-1}^\top), \text{ when } j > 1 \quad (6)$$

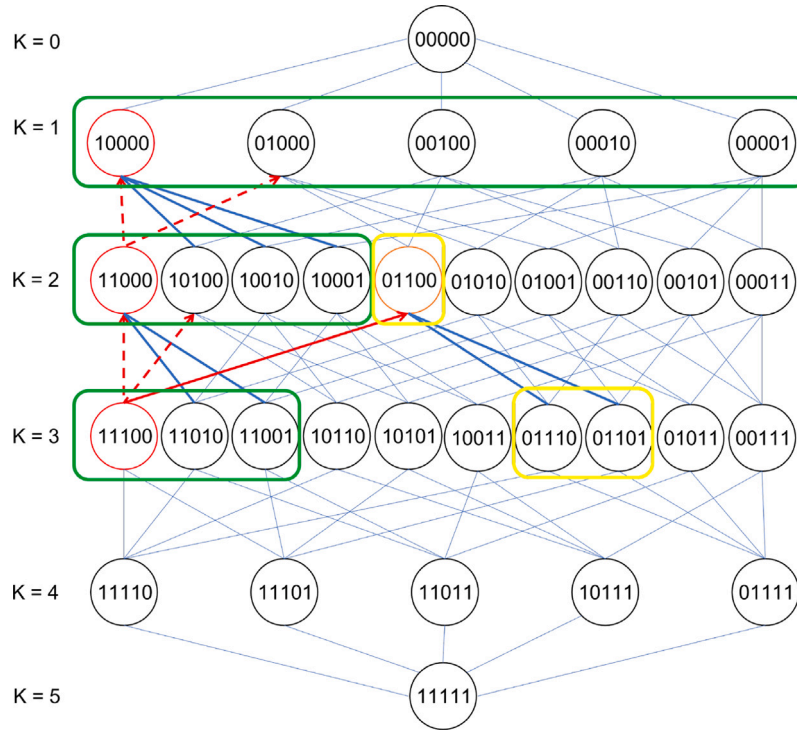


Fig. 2. All possible combinations for any data set with $p = 5$ variables when the selected nonzero variables $K = 1, 2, \dots, p$.

Algorithm 1 $[\mathbf{a}_{p \times q}] = \text{D-SPCA}(\Sigma_{p \times p}, q, M, K_j)$

Require: Covariance matrix $\Sigma_{p \times p}$, number of sensors M , number of sparse PCs q , number of nonzero variables K_j in PC j for $j = 1, 2, \dots, q$, or sparsity parameters α_M, α_K .

- 1: Initialize $k = \mathbf{0}_{1 \times q}$
- 2: Set $K_1 = \text{int}(\alpha_M M)$
- 3: Set $\mathcal{P} = \{x_i | i = 1, 2, \dots, p\}$
- 4: **for** $j = 2$ to q **do**
- 5: $K_j = \text{int}(\alpha_K K_{j-1})$
- 6: **end for**
- 7: **for** $j = 1$ to q **do**
- 8: Initialize $\mathcal{U}_j = \emptyset$
- 9: Set $\mathcal{L}_j = \text{unique}(\bigcup_{i=1}^j \mathcal{U}_i)$
- 10: **while** $k_j \leq K_j$ **do**
- 11: **if** $\text{length}(\mathcal{L}_j) < M$ **then**
- 12: $\text{CV} = \mathcal{P} \setminus \mathcal{U}_j$ $\triangleright$ CV is Candidate Variables set
- 13: **else if** **then**
- 14: $\text{CV} = \mathcal{L}_j \setminus \mathcal{U}_j$
- 15: **end if**
- 16: FVS Adding variables
- 17: BVE Removing variables
- 18: **end while**
- 19: $\text{Var}_j, \mathbf{a}_j = \text{eig}(\Sigma_{(\mathcal{U}_j, \mathcal{U}_j)})$
- 20: $\Sigma_{j+1} = \Sigma_j - \mathbf{a}_j^T \Sigma \mathbf{a}_j (\mathbf{a}_j \mathbf{a}_j^T)$
- 21: **end for**
- 22: **return** $\mathbf{a}_{p \times q}$

The objective function of problem (5) aims to maximize the EV of the j th PC. The first constraint guarantees the loading vector $\mathbf{a}_j$ is a unit vector, the second constraint ensures the sparseness of j th PC is less or equal to the desired number of nonzero variables, and the third constraint control the total number of used variables in the first j PCs are less than the number of available sensors. $\mathcal{L}_j$ is a set of used variables in the first j PCs. $\mathcal{L}_j = \text{unique}(\bigcup_{i=1}^j \mathcal{U}_i)$, where $\mathcal{U}_j$ is a set of used variables in the j th PC.

To find the sparse loading vector for all q PCs, the D-SPCA adopts the iterative procedure elaborated in Algorithm 1. The sequence of ascending order determines the loading vector of each PC. Lines 1–6 initialize the sparsity parameters required by the algorithm and the set of variables $\mathcal{P}$. For the j from 1 to q , $\mathcal{U}_j$, a set of used variables in the j th PC, is initialized. Then, the set of variables used in the first j PCs $\mathcal{L}_j$ is updated in line 9. When k_j , the number of variables used in the j th PC, is less than the preset value K_j , in lines 11–15, a set of candidate variables is calculated using $\mathcal{P}$ or $\mathcal{L}_j$ and $\mathcal{U}_j$ depending on the number of elements used in the variables set $\mathcal{L}_j$. Depending on the sets of candidate variables and used variables, the FVS and BVE methods are used to add variables to the set $\mathcal{U}_j$ and remove variables from the set $\mathcal{U}_j$, respectively. Line 19, SVD is implemented to calculate the EV Var_j and the loading vector $\mathbf{a}_j$ of the j th PC. Then, the deflated covariance matrix is updated using (6). Eventually, q sparse loading vectors are generated and presented as a matrix $\mathbf{a}_{p \times q}$. Fig. 3 presents the flowchart of Algorithm 1.

3.2. Forward variable selection method

Algorithm 2 explains the process for adding variables to set $\mathcal{U}_j$. Given the candidate variables set CV, the number of variables used in j th PC k_j and set of used variables $\mathcal{U}_j$, Algorithm 2 updates k_j and $\mathcal{U}_j$, and calculates the EV Var_j after the selected variable is added to the set. Initialize an empty elimination set $\mathcal{V}_j$ for the j th PC in line 1. In lines 2–6, every variable i in the set CV is added to the set $\mathcal{U}_j$ to generate a new set l . Then, the subcovariance matrix $\Sigma_{(l,l)}$ is extracted and the largest eigenvalue across all generated sets l is calculated. The calculated eigenvalue is recorded in the variance set $\mathcal{V}_j$. Line 7 seeks the index that provides the largest variance. Then, lines 8 and 9 update the set $\mathcal{U}_j$ and increase k_j by one.

3.3. Backward variable elimination method

When k_j , that is, the number of variables used in the j th PC, is greater than two, the Algorithm 3 is activated to ensure that more

Algorithm 2 $[k_j, \mathcal{U}_j] = \text{FVS}(CV, k_j, \mathcal{U}_j)$

Require: Candidate Variables set $CV, k_j, \mathcal{U}_j$

- 1: Initialize $\mathcal{V}_j = \emptyset$
- 2: **for** i in CV **do**
- 3: $l = \mathcal{U}_j \cup \{i\}$
- 4: $\text{eigvals}, \text{eigvecs} = \text{eig}(\Sigma_{(l,l)})$
- 5: $\mathcal{V}_j.\text{append}(\text{eigvals})$
- 6: **end for**
- 7: $\text{index} = \text{argmax}(\mathcal{V}_j[\text{index}])$
- 8: $\mathcal{U}_j = \mathcal{U}_j \cup CV[\text{index}]$
- 9: $k_j = k_j + 1$
- 10: **return** $k_j, \mathcal{U}_j$

Algorithm 3 $[k_j, \mathcal{U}_j] = \text{BVE}(\mathcal{U}_j, k_j, \text{Var}_j)$

Require: $\mathcal{U}_j, k_j, \text{Var}_j$

- 1: **while** $k_j > 2$ **do**
- 2: Initialize $\mathcal{V}_j = \emptyset$
- 3: **for** i in $\mathcal{U}_j$ **do**
- 4: $l = \mathcal{U}_j \setminus \{i\}$
- 5: $\text{eigvals}, \text{eigvecs} = \text{eig}(\Sigma_{(l,l)})$
- 6: $\mathcal{V}_j.\text{append}(\text{eigvals})$
- 7: **end for**
- 8: $\text{index} = \text{argmax}(\mathcal{V}_j[\text{index}])$
- 9: **if** $\max(\mathcal{V}_j) > \text{Var}_j$ **then**
- 10: $\mathcal{U}_j = \mathcal{U}_j \setminus \mathcal{U}_j[\text{index}]$
- 11: $k_j = k_j - 1$
- 12: **end if**
- 13: **end while**
- 14: **return** $k_j, \mathcal{U}_j$

variable combinations are considered. Initialize an empty variance set $\mathcal{V}_j$ for j th PC elimination step in line 2. For every variable i in set $\mathcal{U}_j$, it removes the variable i from $\mathcal{U}_j$ to create a new set l ; then, it constructs the subcovariance matrix $\Sigma_{(l,l)}$, determines the largest eigenvalue across all generated sets l , and records the calculated eigenvalue in the variance set $\mathcal{V}_j$. Line 8 searches for the index that offers the largest variance. In lines 9–12, if the largest variance value in set $\mathcal{V}_j$ is greater than the variance Var_j , the variable in the index of set $\mathcal{U}_j$ is removed, set $\mathcal{U}_j$ is updated, and k_j is decreased by one.

3.4. Computational complexity

The computational complexity of the algorithms for each PC differs depending on the size of CV and $\mathcal{U}_j$. When calculating the first PC, the computational complexity of step 19 and 20 in Algorithm 1 is $O(K_j^3 + 3p^2)$, the computational complexity of Algorithm 2 for all k_j is approximately $O(p(\frac{(K_j+1)(K_j+2)}{2})^2 + p)$, and the computational complexity of Algorithm 3 for all k_j is approximately $O(K_j(\frac{K_j(K_j+1)}{2})^2 + K_j)$. Because $K_j \leq p$, the overall computational complexity of the first PC is dominated by Algorithm 2, which is $O(p(\frac{(K_j+1)(K_j+2)}{2})^2 + p)$.

For the second and subsequent PCs, the computational complexity of the algorithms is the same as that for the first PC until the number of elements in set $\mathcal{L}_j$ reaches the number of total variables allowed M . When the number of variables used is M , the computational complexity of Algorithm 2 changes to $O(M(\frac{(K_j+1)(K_j+2)}{2})^2 + M)$, while the computational complexity of step 19 and 20 in Algorithm 1 and that of Algorithm 3 for all k_j remain the same. Because $K_j \leq M$, the overall computational complexity of the PCs is still dominated by Algorithm 2, which is $O(M(\frac{(K_j+1)(K_j+2)}{2})^2 + M)$. Thus, for a total of q PCs, the approximate computational complexity of the D-SPCA algorithm is between $O(qMK_j^4)$ and $O(qpK_j^4)$. Compared to the method that solves the combinatorial optimization problem by using enumeration searching with the computational complexity $O(p^{\min(K_j, p-K_j)})$ for each PC, the D-SPCA, which uses the greedy approach, finds a sub-optimal solution within a significantly faster time.

4. Experiments

In this section, to verify the performance of the proposed D-SPCA method, experiments are conducted using two synthetic datasets and a real-world dataset related to semiconductor manufacturing. The numerical results obtained in this manner are compared to those obtained using the two benchmark methods, namely sparse PCA (SPCA) (Zou et al., 2006) and preserved pattern SPCA (PP-SPCA) (Seghouane et al., 2019).

4.1. Synthetic example

To illustrate that the proposed method effectively and efficiently reduces the number of variables used while maintaining the total EV, two modified synthetic examples proposed by Zou et al. (2006) are used. In their study, Zou et al. (2006) employed three zero-mean hidden factors with variances of 290, 300, and 283.8, respectively, to generate a total of 10 variables. Similarly, in our experiment, we adopt the three hidden factors utilized by Zou et al. (2006) and introduce an additional hidden factor to enhance the diversity of the data. The specific results are as follows:

4.1.1. Low-dimensional example with 16 variables

This synthetic example involves four hidden factors, of which two are independent and follow normal distributions with zero mean and variances of 290 and 300, respectively:

$$V_1 \sim N(0, 290), \quad V_2 \sim N(0, 300).$$

The third and fourth factors are linear combinations of the first two factors with independent noise:

$$V_3 = -0.3V_1 + 0.925V_2 + \epsilon, \quad \epsilon \sim N(0, 1).$$

$$V_4 = 0.925V_1 - 0.3V_2 + \epsilon, \quad \epsilon \sim N(0, 1).$$

The coefficients follow the settings in Zou et al. (2006). Utilizing these four hidden factors, 16 input variables are constructed as follows:

$$x_i = V_j + \epsilon_i^j, \quad \epsilon_i^j \sim N(0, 1)$$

where j represents the j th hidden factor, and i ranges from 1 to 4 when $j = 1$, i ranges from 5 to 8 when $j = 2$, i ranges from 9 to 12 when $j = 3$, and i ranges from 13 to 16 when $j = 4$. The ϵ_i^j terms are independent. The exact covariance matrix is utilized to compute the principal components.

The main goal of PCA and sparse PCA is maximizing the EV. Owing to the nature of the formulations of the most sparse PCA methods, the extracted PCs can be correlated; thus, when the total EV is calculated by tracing the transformed covariance matrix, the total EV is overestimated. Therefore, in this study, the adjusted EV is calculated using the method proposed in Zou et al. (2006) to eliminate such correlations. To increase the interpretability of each extracted feature, ensuring the sparsity of the loading vector is an essential objective in sparse PCA. The performance measures in this section are adjusted EV, cumulative adjusted EV, number of nonzero variables used in each PC, and total number of variables used in the first j PCs.

Table 2 summarizes the result of the first six PCs for all three methods. All the methods, except PP-SPCA, produced similar results, which explained about 57% of the total variance. However, SPCA eventually used all 16 variables in the first 6 PCs. Although the PP-SPCA method used only some of the original variables in the extracted features, the preserved sparse pattern in PP-SPCA forced all the PCs to consist of the same variables that eliminate the diversity of interpretation. Moreover, when more variables were employed, PP-SPCA returned less total EV than D-SPCA. Despite D-SPCA being a sub-optimal method, the solution indicated that the EV in the first 6 PCs was very extremely to that of the SPCA method and superior to that of the PP-SPCA method.

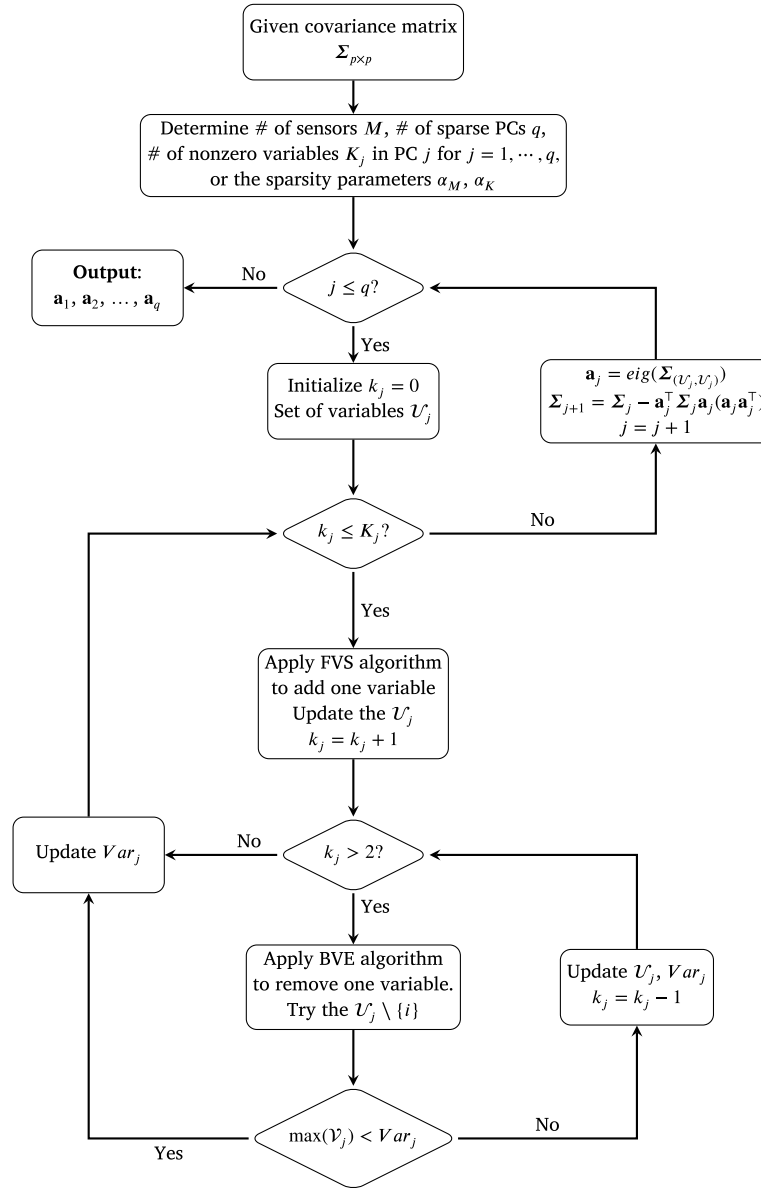


Fig. 3. Flowchart of the dynamic sparse PCA algorithm.

Table 2
Performance comparison of the methods for low-dimensional synthetic data.

Methods	Measurement	Principal component					
		PC1	PC2	PC3	PC4	PC5	PC6
SPCA	Explained variance (%)	29.22	28.18	0.06	0.04	0.03	0.03
	Cum. Ex. variance (%)	29.22	57.40	57.46	57.50	57.53	57.56
	# of nonzero variables	4	4	4	4	4	4
	# of variables used	4	8	12	14	15	16
PP-SPCA	Explained variance (%)	36.97	16.58	0.05	0.02	0.02	0.01
	Cum. Ex. variance (%)	36.97	53.55	53.60	53.62	53.64	53.65
	# of nonzero variables	10	10	10	10	10	10
	# of variables used	10	10	10	10	10	10
D-SPCA	Explained variance (%)	29.22	28.18	0.01	0.01	0.01	0.01
	Cum. Ex. variance (%)	29.22	57.40	57.41	57.43	57.44	57.45
	# of nonzero variables	4	4	4	4	4	4
	# of variables used	4	8	8	8	8	8

Table 3
Performance comparison of the methods for high-dimensional synthetic data.

Methods	Measurement	Principal component			
		PC1	PC2	PC3	PC4
SPCA	Explained variance (%)	28.32	27.80	0.04	0.00
	Cum. Ex. variance (%)	28.32	56.12	56.16	56.16
	# of nonzero variables	400	400	400	400
	# of variables used	400	800	1128	1352
PP-SPCA	Explained variance (%)	35.92	21.31	0.02	0.00
	Cum. Ex. variance (%)	35.92	57.23	57.25	57.25
	# of nonzero variables	1085	1085	1085	1085
	# of variables used	1085	1085	1085	1085
D-SPCA	Explained variance (%)	29.20	28.15	0.00	0.00
	Cum. Ex. variance (%)	29.20	57.35	57.35	57.35
	# of nonzero variables	400	400	400	400
	# of variables used	400	800	800	800

4.1.2. High-dimensional example with 1600 variables

To illustrate that the proposed method can solve large-scale problems, a high-dimensional example was developed. Similar to the low-dimensional example in the previous subsection, the high-dimensional synthetic example contained four hidden factors. Two of them were independent and followed normal distributions:

$$V_1 \sim N(0, 290), \quad V_2 \sim N(0, 300).$$

The third and fourth factors were linear combinations of the first two factors with independent noise:

$$V_3 = -0.3V_1 + 0.925V_2 + \epsilon, \quad \epsilon \sim N(0, 1).$$

$$V_4 = 0.1V_1 + 0.7V_2 + \epsilon, \quad \epsilon \sim N(0, 1).$$

By using these four hidden factors, 1600 input variables were constructed as follows to demonstrate the scalability of the D-SPCA method in a high-dimensional case.

$$x_i = V_j + \epsilon_i^j, \quad \epsilon_i^j \sim N(0, 1)$$

where j represents the j th hidden factor, and i ranges from 1 to 400 when $j = 1$, i ranges from 401 to 800 when $j = 2$, i ranges from 801 to 1200 when $j = 3$, and i ranges from 1201 to 1600 when $j = 4$; and ϵ_i^j are independent. The exact covariance matrix was used to compute the principal components.

As shown in Table 3, the methods produced similar results, explaining about 57% of the total variance. However, SPCA and PP-SPCA used a total of 1352 and 1085 variables in the first four PCs, which are 69% and 35% higher, respectively, than the number of variables used in D-SPCA. The proposed D-SPCA method exhibited the best sparsity among all the benchmark methods, as well as a competitive variance extraction property. Additionally, the proposed method achieved sparsity with only half of the original variables in the extracted features in this example. In practical engineering problems, the number of variables is often related to the number of sensors. In Section 4.2, a real-world example from the semiconductor area is used to illustrate that reducing the number of variables can lead to a reduction in sensor cost.

4.2. Case study: Etching process in semiconductor manufacturing

To validate whether the proposed D-SPCA method is suitable for the VM model, a semiconductor manufacturing dataset is used in the experiments. The values of EV obtained using the benchmark methods and the proposed method are compared after dimensionality reduction, and the performance levels of the prediction models with different numbers of extracted features calculated using various dimensionality reduction methods are evaluated.

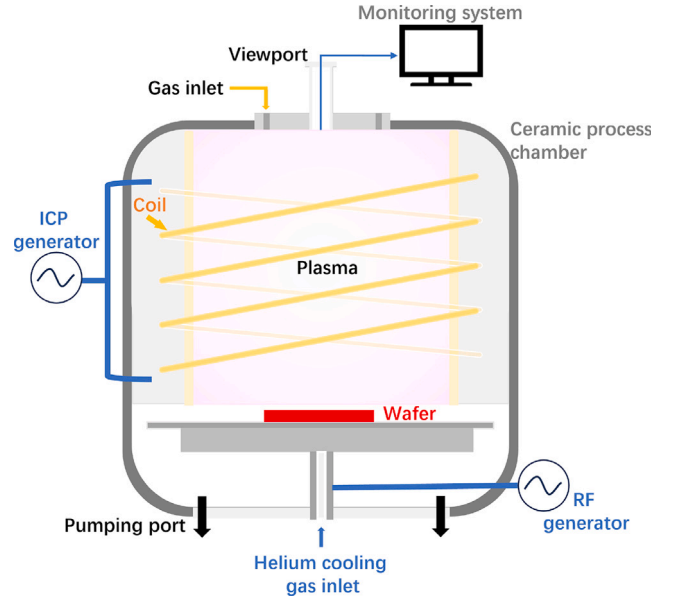


Fig. 4. Example of inductively coupled plasma system.

4.2.1. Dataset description

Etching is a crucial procedure in semiconductor production. The complex physical and chemical reactions in the etching process directly affect the critical dimensions (CD) and yield of semiconductors. Fig. 4 illustrates an inductively coupled plasma (ICP) system. After the etching gas is injected into the etching chamber, a radio frequency (RF) current is applied to generate plasma. The sensors record the information that affects critical dimensions, such as gas flow speed, temperature, RF voltage, and chamber pressure.

The data from these sensors is presented in a time-series format. Thus, utilizing the raw data from each sensor directly as inputs for the model would result in thousands of variables, yielding an inefficient approach for the model construction. The etching process comprises multiple sub-steps, and the data obtained in this experiment originates from one of its crucial stages. To effectively manage the time-series data obtained from sensors, we treat each sensor as a variable and employ the following method to map the data from each sensor to its corresponding variable value. We compute the average value of the data generated by each sensor during this stage. These average values are then utilized as representative values for the respective variables associated with each sensor.

The dataset utilized in the experiment was collected from a plasma etching process within semiconductor manufacturing. Each observation corresponds to a single wafer, with data monitored by 36 sensors. Following data preprocessing, which involved eliminating outliers and converting the time series data from each sensor during the critical stage into representative values represented by a single variable, a total of 75 observations remained. For each observation, the values of 36 variables were derived from the 36 sensors listed in Table 4. These sensors primarily monitor chamber pressure, various gas flow rates, RF intensity, and temperatures across different zones. The output variable was the critical dimension obtained from physical metrology. A VM model was constructed using the sensor data and physical metrology results from the 75 observations.

4.2.2. Dimensionality reduction for VM model

The D-SPCA and the two benchmark methods were used to reduce the dimensions of the original input data. The extracted features were used as the input variables for the VM model. The percentage of the variance explained by the sparse PCs was determined in advance to

Table 4
List of used sensors with name.

#	Sensor name	#	Sensor name
1	Chamber Base Press	19	MFC01 C4F6 50
2	Dcbias Voltage	20	MFC02 C4F8 50
3	He Inner Flow	21	MFC03 Ar 2000
4	He Outer Flow	22	MFC04 CF4 300
5	Chamber Leakrate	23	MFC05 CHF3 20
6	Confinement Ring Position	24	MFC06 CH2F2 50
7	Dual Gas Adjust Press	25	MFC07 CHF3 100
8	FFU Left RPM	26	MFC08 N2 500
9	FFU Right RPM	27	MFC09 O2 20
10	RF 2MHz Reflected Power	28	MFC10 O2 100
11	RF 60MHz Reflected Power	29	MFC11 O2 3000
12	TM Press	30	MFC12 CH4 200
13	Bottom Temp	31	OES IB 1
14	Chamber Press	32	RF 60 MHz Forward Power
15	Differential Press	33	Top Plate Temp
16	ESC Voltage	34	Top Temp
17	Foreline Press	35	Tunning Gas Flow
18	He Inner Press	36	Process Step Time

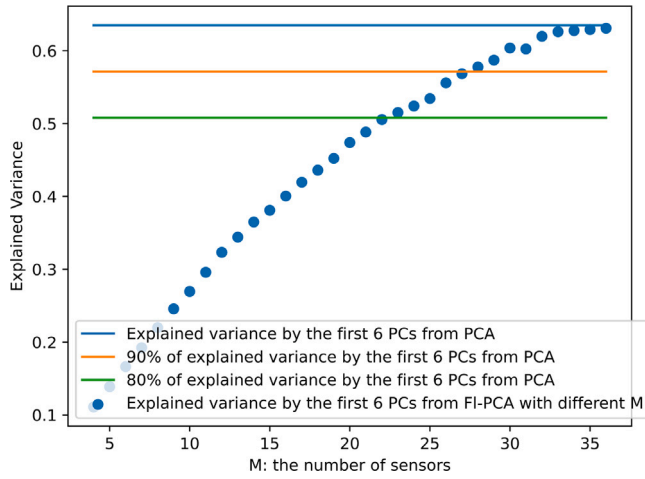


Fig. 5. Explained variance by different M .

select the appropriate M . In Fig. 5, the first six PCs obtained from PCA account for 63% of the total variance, as indicated by the solid blue line. The green and solid red lines represent 90% and 80% of the total EVs of the first six PCs obtained by PCA, respectively. The blue dotted line illustrates the variance explained by the first six PCs obtained using FI-PCA with different numbers of sensors M . According to the figure, when the number of sensors M was 22, the D-SPCA method was able to identify the sparse PCs that explained 80% of the variance from PCA.

After determining the total number of sensors M used in the VM model, an appropriate K_i must be selected for the i th PC. The blue dotted line in Fig. 6 represents the cumulative EV for K_1, K_2, K_3, K_4 in the first four PCs. The solid blue line represents the adjusted EV when the number of nonzero variables K_i in the i th PC is the same as K_{i-1} in the $i-1$ PC because the number of nonzero variables and EV generally follow a decreasing order. The red and green solid lines represent 95% and 90%, respectively, of the EV. Plot (a) shows the adjusted EV of the first PC with different K_1 when $M = 22$. K_1 is set to 20 to obtain 96% of the EV to balance sparsity and EV. Plot (b) indicates the adjusted EV of the first two PCs with different K_2 when $K_1 = 20$. K_2 is set to 14 to achieve 99% of the EV. Plot (c) expresses the adjusted EV of the first three PCs with different K_3 when $K_2 = 14$. K_3 is set to 11 to achieve 99% of the EV. Plot (d) is the adjusted EV of the first four PCs with different K_4 when $K_3 = 11$.

The performance measures are the same as those for the synthetic examples in Section 4.1. The results of all three methods are presented in Table 5. Assume that the total EV and the total number of sensors

used can be regarded as a “bi-objective optimization problem”. The D-SPCA method dominates the other two methods because the SPCA and PP-SPCA methods perform worse in both aspects, namely low EV and a high number of sensors used. The proposed D-SPCA method provides the highest EV while using the least number of sensors. Moreover, it provides the best sparsity and good variance extraction properties. In engineering problems, such as semiconductor manufacturing, it is more critical to reduce sensor costs and provide interpretable results. Thus, the proposed method is superior to the other sparse PCA methods in this example.

4.2.3. VM prediction model

The extracted features form the data matrix $\mathbf{Z}_{n \times q}$, which is used as the input variables in the regression or classification VM model. This can be calculated using (7):

$$\mathbf{Z}_{n \times q} = \mathbf{X}_{n \times p} \mathbf{a}_{p \times q} \quad (7)$$

where $\mathbf{X}$ represents the original input data matrix and $\mathbf{a}$ is the loading vector computed using the sparse PCA method. Multiple linear regression (MLR), a popular statistical method, was used as the prediction model in the experiment owing to its interpretability and simplicity. This method is generally used to predict the output value, which is a linear combination of the input variables, and it does not require any parameter.

The total number of observations was 75. The three-fold cross-validation (CV) method was used to divide the observations into three folds containing 25 observations each. In each experiment, the data were evenly divided into three folds, where two of the folds were selected as the training set and the remaining fold as the testing set. The process was repeated thrice to complete one experiment. The dimensionality reduction technique was applied to the training dataset to determine the loading vectors $\mathbf{a}$ and the extracted features $\mathbf{Z}_{train}$ before building the prediction model. The MLR model seeks a set of coefficients β that minimizes the Euclidean distance between the true values $\mathbf{Y}_{train}$ and the predicted values $\mathbf{Z}_{train}\beta$. The coefficient β can be calculated using (8).

$$\hat{\beta} = \arg \min_{\beta} \|\mathbf{Y}_{train} - \mathbf{Z}_{train}\beta\|^2 \quad (8)$$

For the testing dataset, the original input variables were transformed to the features $\mathbf{Z}_{test}$ by using the loading vectors $\mathbf{a}$. Then, the predicted values $\hat{\mathbf{Y}}_{test}$ were calculated through linear combination of the features $\mathbf{Z}_{test}$ with the coefficients β obtained using the MLR model. That is $\hat{\mathbf{Y}}_{test} = \mathbf{Z}_{test}\beta$ in matrix form. The difference between the predicted values $\hat{\mathbf{Y}}_{test}$ and the true values $\mathbf{Y}_{test}$ was used to calculate various performance measures. For instance, root mean square error (RMSE) and mean absolute error (MAE) are widely used in machine learning to evaluate the prediction accuracy of different methods.

Two critical dimensions were predicted in the experiments. The results presented in Table 6 were calculated from the average of 10 experiments with three-fold CV. Compared to the benchmark methods, the proposed method had the worst RMSE but the best MAE in both CD1 and CD2 for the training and testing data. However, in terms of the p -value, the statistical test yielded no significant difference between the RMSEs obtained using different methods. Thus, the prediction performance of the proposed method was similar to that of the other sparse PCA methods, but the proposed method used a considerably smaller number of variables.

4.3. Discussion

The results of the experiments indicate that, compared to benchmark methods, our proposed D-SPCA method not only reduces data acquisition costs but also ensures the interpretability of the extracted PCs and the scalability of the algorithm. In semiconductor manufacturing processes, data acquisition costs primarily stem from the purchase

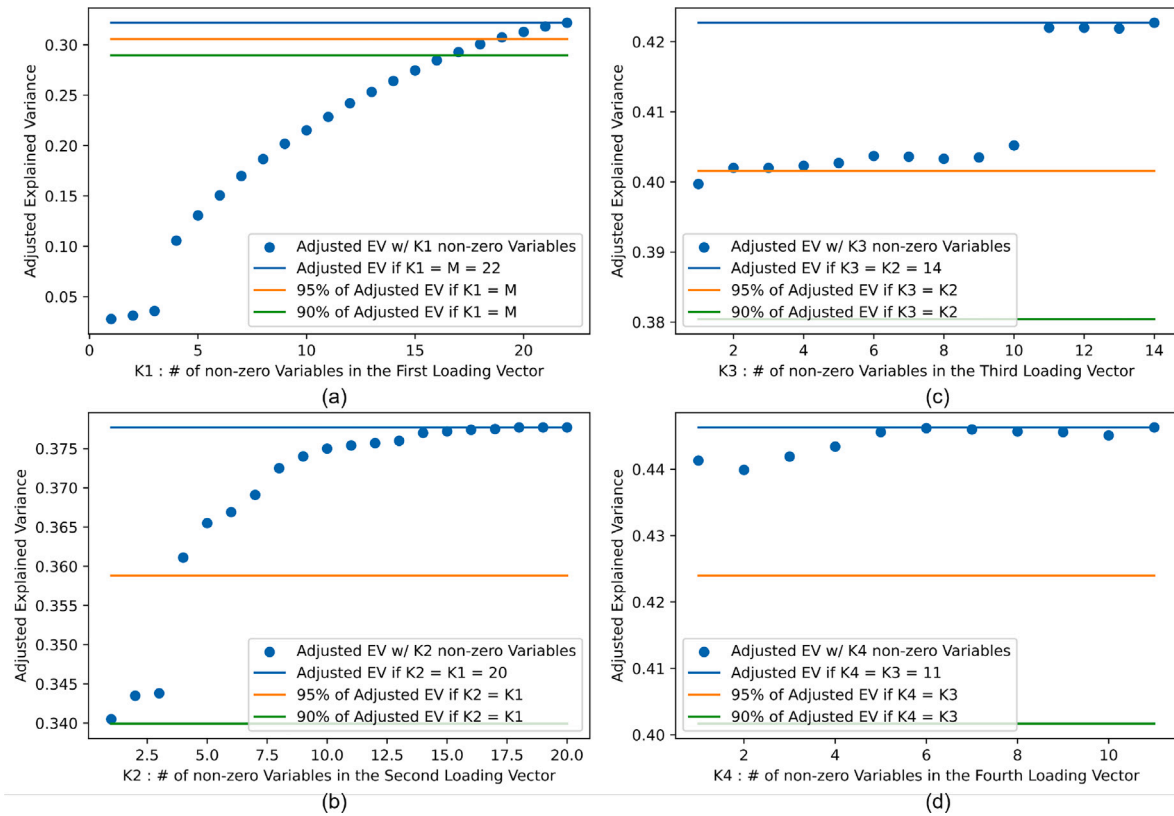


Fig. 6. Cumulative EV with different K_1, K_2, K_3 , and K_4 . (a) Adjusted EV of the first PC with different K_1 when $M = 22$. (b) Adjusted EV of the first two PCs with different K_2 when $K_1 = 20$. (c) Adjusted EV of the first three PCs with different K_3 when $K_2 = 14$. (d) Adjusted EV of the first four PCs with different K_4 when $K_3 = 11$.

Table 5

Performance comparison using real-world data.

Methods	Measurement	Principal component					
		PC1	PC2	PC3	PC4	PC5	PC6
SPCA	Explained variance (%)	13.25	6.73	5.3	4.98	2.78	4.16
	Cum.Ex. variance (%)	13.25	19.98	25.28	30.26	33.04	37.2
	# of nonzero variables	20	14	11	6	6	6
	& # of sensors used	20	28	28	30	32	33
PP-SPCA	Explained variance (%)	26.48	6.2	5.25	4.39	3.74	3.2
	Cum.Ex. variance (%)	26.48	32.68	37.93	42.32	46.06	49.26
	# of nonzero variables	25	25	25	25	25	25
	& # of sensors used	25	25	25	25	25	25
D-SPCA	Explained variance (%)	31.28	6.42	4.5	3	2.42	2
	Cum.Ex. variance (%)	31.28	37.7	42.2	45.2	47.62	49.62
	# of nonzero variables	20	14	11	6	6	6
	& # of sensors used	20	22	22	22	22	22

Table 6

RMSE and MAE comparison for real-world data.

Method	RMSE				MAE			
	Mean		Variance		Mean		Variance	
	Train	Test	Train	Test	Train	Test	Train	Test
CD1								
SPCA	0.881	0.962	0.004	0.009	0.833	0.889	0.006	0.012
PP-SPCA	0.887	0.968	0.004	0.010	0.837	0.899	0.006	0.012
D-SPCA	0.888	0.977	0.003	0.010	0.828	0.884	0.006	0.015
CD2								
SPCA	0.693	0.764	0.003	0.007	0.643	0.689	0.003	0.008
PP-SPCA	0.700	0.768	0.003	0.009	0.645	0.699	0.003	0.009
D-SPCA	0.702	0.774	0.003	0.008	0.634	0.683	0.003	0.009

of sensors. Semiconductor production lines are equipped with hundreds of sensors that continuously collect data crucial for monitoring and enhancing production efficiency. These sensors, strategically placed at various stages of processing, play a vital role in ensuring the quality, performance, and maximum yield of semiconductor products. However, according to literature sources,¹ there is a projected persistent demand for sensors accompanied by price increases. For instance, in 2021, the average sensor price was \$0.41, with an expected rise to \$0.44 in 2022. Furthermore, the high dimensionality of data leads to an exponential increase in data volume. Therefore, reducing the number of sensors while maintaining prediction accuracy in semiconductor manufacturing could significantly contribute to increasing profitability in the semiconductor industry.

¹ <https://electronics-sourcing.com/2022/06/29/sensor-demand-will-remain-strong-and-prices-will-rise/>.

Additionally, in our case study experiment, the deployment of our novel D-SPCA methodology offers insights into the dataset derived from sensor data. Through sparse analysis results and based on the names in Table 4, we identify the significant roles of various sensor categories in the VM model. For instance, considering the first few PCs, PC1 primarily captures variance associated with the environmental and operational aspects of the semiconductor etching process. It encompasses sensors related to pressure, flow, and mass flow controllers (MFC) for managing gases such as hexafluorobutadiene (C4F6). PC1 underscores the essential parameters of gas flow rates and chamber pressures crucial for successful semiconductor fabrication. PC2 emphasizes the importance of air quality within the cleanroom, with significant contributions from fan filter unit (FFU) sensors. This highlights the stringent requirements for air quality management and the sensitivity of the process to atmospheric purity. PC3 focuses on gas flow management, spotlighting MFC sensors regulating nitrogen and oxygen flow. This underscores the critical role of gas composition in semiconductor process outcomes. PC4 highlights the importance of maintaining precise pressures within the etching chamber, as indicated by sensors measuring helium pressure. This underscores the necessity of exact pressure control for optimal process conditions.

Moreover, the computational complexity from Section 3.4 and the results from Section 4.1.2 demonstrate that the proposed D-SPCA exhibits excellent scalability, effectively reducing the dimensionality of data with thousands of variables and decreasing the number of used variables. This aligns with the variable counts typically encountered in practical production scenarios, thus rendering D-SPCA suitable for various industrial settings.

5. Conclusion

In this paper, we propose a novel PCA method with a dynamic sparse algorithm called the D-SPCA method. This method is designed to emphasize global sparsity in the first few PCs. Our method ensures that the extracted features utilized in the VM model comprise a limited number of variables, thereby enhancing the interpretability of the model and reducing sensor costs. The efficacy of our proposed approach is validated through experimental evaluations conducted on two synthetic datasets and one real-world case study. Compared to conventional sparse PCA techniques that employ all input variables derived from the extracted features, our method specifically focuses on selecting features with a limited number of input variables, which achieves a distinct advantage in terms of global sparsity, a highly desirable property in the context of VM models. In summary, our method effectively utilizes the proposed dimensionality reduction approach to address the substantial influx of data from IoT in semiconductor manufacturing and apply the data in the proposed VM model. It enables real-time monitoring to minimize defects and costs and offers a means to optimize sensor usage, thereby contributing to improved operational efficiency and profitability in the industry.

The limitations of the proposed method are as follows. Similar to most sparse PCA methods, the orthogonal property between the loading vectors obtained using the proposed method is ignored. Additionally, while feature variability is significant as a crucial indicator in PCA methods, it does not directly translate to predictive performance. Therefore, exploring a supervised version of D-SPCA could be a fruitful direction for future research, allowing for a more nuanced understanding of how feature extraction impacts prediction in virtual manufacturing. Furthermore, without considering sensor costs, the sparsity parameter M , which determines the total number of nonzero variables in principal components, cannot be optimized mathematically. The current dynamic method utilizes only a single selection and elimination approach; developing more forward selection and backward removal methods could accelerate the addition process and enhance potential reasonable solutions.

CRedit authorship contribution statement

Tianhui Wang: Methodology, Software, Validation, Formal analysis, Writing – original draft, Writing – review & editing, Visualization, Investigation. **Yifan Xie:** Methodology, Software, Validation, Investigation, Writing – original draft, Visualization. **Young-Seon Jeong:** Writing – review & editing, Conceptualization, Methodology. **Myong K. Jeong:** Writing – review & editing, Supervision, Conceptualization, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors thank the reviewers for the helpful and constructive comments that greatly contributed to the improvement of the paper. This work was supported in part by National Research Foundation of Republic of Korea Grant (NRF-2022R1F1A1063174).

References

- Abdi, H. (2007). Singular value decomposition (SVD) and generalized singular value decomposition. *Encyclopedia of Measurement and Statistics*, 907(912), 44.
- Adly, F., Alhussein, O., Yoo, P. D., Al-Hammadi, Y., Taha, K., Muhaidat, S., Jeong, Y.-S., Lee, U., & Ismail, M. (2015). Simplified subspace regression network for identification of defect patterns in semiconductor wafer maps. *IEEE Transactions on Industrial Informatics*, 11(6), 1267–1276.
- Arima, S., Nagata, T., Bu, H., & Shimada, S. (2019). Applications of sparse modelling and principle component analysis for the virtual metrology of comprehensive multi-dimensional quality. In *ICORES* (pp. 354–361).
- Berk, L., & Bertsimas, D. (2019). Certifiably optimal sparse principal component analysis. *Mathematical Programming Computation*, 11, 381–420.
- Cadima, J., & Jolliffe, I. T. (1995). Loading and correlations in the interpretation of principle components. *Journal of Applied Statistics*, 22(2), 203–214.
- Corallo, A., Crespino, A. M., Del Vecchio, V., Lazoi, M., & Marra, M. (2021). Understanding and defining dark data for the manufacturing industry. *IEEE Transactions on Engineering Management*.
- d'Aspremont, A., Bach, F., & El Ghaoui, L. (2008). Optimal solutions for sparse principal component analysis. *Journal of Machine Learning Research*, 9(7).
- d'Aspremont, A., Ghaoui, L., Jordan, M., & Laffont, G. (2004). A direct formulation for sparse PCA using semidefinite programming. *Advances in Neural Information Processing Systems*, 17.
- Gou, J., Yuan, X., Xue, Y., Du, L., Yu, J., Xia, S., & Zhang, Y. (2023). Discriminative and geometry-preserving adaptive graph embedding for dimensionality reduction. *Neural Networks*, 157, 364–376.
- Guillemot, V., Beaton, D., Gloaguen, A., Löfstedt, T., Levine, B., Raymond, N., Tenenhaus, A., & Abdi, H. (2019). A constrained singular value decomposition method that integrates sparsity and orthogonality. *PLoS One*, 14(3), Article e0211463.
- Jolliffe, I. T. (1995). Rotation of principal components: choice of normalization constraints. *Journal of Applied Statistics*, 22(1), 29–35.
- Jolliffe, I. T. (2002). *Principal component analysis for special types of data*. Springer.
- Jolliffe, I. T., Trendafilov, N. T., & Uddin, M. (2003). A modified principal component technique based on the LASSO. *Journal of Computational and Graphical Statistics*, 12(3), 531–547.
- Kang, P., Lee, H.-j., Cho, S., Kim, D., Park, J., Park, C.-K., & Doh, S. (2009). A virtual metrology system for semiconductor manufacturing. *Expert Systems with Applications*, 36(10), 12554–12561.
- Kwak, N. (2014). Principal component analysis by L_p -norm maximization. *IEEE Transactions on Cybernetics*, 44(5), 594–609.
- Li, X., Wang, Y., & Ruiz, R. (2022). A survey on sparse learning models for feature selection. *IEEE Transactions on Cybernetics*, 52(3), 1642–1660.
- Liu, Y., Zeng, J., Xie, L., Luo, S., & Su, H. (2018). Structured joint sparse principal component analysis for fault detection and isolation. *IEEE Transactions on Industrial Informatics*, 15(5), 2721–2731.
- Mackey, L. (2008). Deflation methods for sparse PCA. *Advances in Neural Information Processing Systems*, 21.
- Moghaddam, B., Weiss, Y., & Avidan, S. (2005). Spectral bounds for sparse PCA: Exact and greedy algorithms. *Advances in Neural Information Processing Systems*, 18.
- Nguyen, C. M., Li, X., Blanton, R. D., & Li, X. (2019). Partial Bayesian co-training for virtual metrology. *IEEE Transactions on Industrial Informatics*, 16(5), 2937–2945.

- Pan, T.-H., Sheng, B.-Q., Wong, D. S.-H., & Jang, S.-S. (2011). A virtual metrology system for predicting end-of-line electrical properties using a MANCOVA model with tools clustering. *IEEE Transactions on Industrial Informatics*, 7(2), 187–195.
- Pearson, K. (1901). LIII. On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11), 559–572.
- Qiu, Y., Lei, J., & Roeder, K. (2023). Gradient-based sparse principal component analysis with extensions to online learning. *Biometrika*, 110(2), 339–360.
- Ren, J., & Ni, D. (2021). A real-time monitoring framework for wafer fabrication processes with run-to-run variations. *IEEE Transactions on Semiconductor Manufacturing*, 34(4), 483–492.
- Richtárik, P., Jahani, M., Ahipasaoglu, S. D., & Takáč, M. (2021). Alternating maximization: unifying framework for 8 sparse PCA formulations and efficient parallel codes. *Optimization and Engineering*, 22(3), 1493–1519.
- Seghouane, A.-K., Shokouhi, N., & Koch, I. (2019). Sparse principal component analysis with preserved sparsity pattern. *IEEE Transactions on Image Processing*, 28(7), 3274–3285.
- Susto, G. A., & Beghi, A. (2013). A virtual metrology system based on least angle regression and statistical clustering. *Applied Stochastic Models in Business and Industry*, 29(4), 362–376.
- Vines, S. (2000). Simple principal components. *Journal of the Royal Statistical Society. Series C. Applied Statistics*, 49(4), 441–451.
- Xia, Z., Chen, Y., & Xu, C. (2021). Multiview pca: A methodology of feature extraction and dimension reduction for high-order data. *IEEE Transactions on Cybernetics*.
- Zeng, D., & Spanos, C. J. (2009). Virtual metrology modeling for plasma etch operations. *IEEE Transactions on Semiconductor Manufacturing*, 22(4), 419–431.
- Zou, H., Hastie, T., & Tibshirani, R. (2006). Sparse principal component analysis. *Journal of Computational and Graphical Statistics*, 15(2), 265–286.
- Zou, H., & Xue, L. (2018). A selective overview of sparse principal component analysis. *Proceedings of the IEEE*, 106(8), 1311–1320.